Your Sidekiq Jobs Have Become a Workflow Engine

BRUG · Sunday, 6 September 2026

Two questions

1. Who runs Sidekiq in production?

2. Who has a Sidekiq job with a **state machine** inside it?

```
GenerateInvoiceJob.perform_async(invoice.id)
```

This is good code.

Order fulfillment

```ruby
class ProcessOrderJob
  include Sidekiq::Job

  def perform(order_id)
    order = Order.find(order_id)
    charge_payment(order)
    reserve_inventory(order)
    ship(order)
  end
end
```

Three steps. One job. Nobody argues with it in review.

Failure 1

Payment succeeds. Worker dies.

Idempotent: safe to run twice

```ruby
def perform(order_id)
  order = Order.find(order_id)
  charge_payment(order)    unless order.charged?
  reserve_inventory(order) unless order.reserved?
  ship(order)              unless order.shipped?
end
```

Nobody asked for `charged_at`.

That's not information about the order. That's information about **how far along the job got**.

The real fix is an idempotency key

Generate it. Store it. Decide what it’s scoped to.

Another column. Another thing you own.

Failure 2

The warehouse takes two days.

A job that schedules itself

```ruby
class CheckWarehouseJob
  def perform(order_id)
    order = Order.find(order_id)
    return ShipOrderJob.perform_async(order_id) if warehouse_ready?(order)

    CheckWarehouseJob.perform_in(1.hour, order_id)   # 🤞
  end
end
```

That's a timer.

We had no way to wait, so we built waiting out of a scheduler and a queue.

```
enum status: %i[pending charged reserved
                awaiting_warehouse shipped]
```

It’s a program counter.

In your orders table. With an index on it.

I’m not saying the code got ugly. The code is fine.

I’m saying we **introduced a concept**, and nobody decided to.

```
enum status: %i[pending charged reserved
                awaiting_warehouse shipped]
```

Failure 3

The warehouse sends a callback instead.

Token. Dedup table. Race on commit.

Failure 4

We deploy mid-flight.

```
# TODO: remove after migration, Mar 2024
```

Failure 5

The customer cancels.

Compensation: the undo you write yourself.

And then someone writes this

```
# Runs every 10 min. Do NOT delete.
# Ask Anubhav before changing this.
# (Anubhav doesn't work here any more.)
class ReconcileStuckOrdersJob
```

Every company I've worked at has this job.

And `git blame` always says it was me.

```
enum status: %i[pending charging charged charge_failed
                charge_retrying reserved reservation_failed
                awaiting_warehouse warehouse_timed_out
                shipping shipped cancelling cancelled
                refund_pending refunded stuck]
```

What we added

Sidekiq

- retries
- scheduling
- queues

Rails / Postgres

- state columns
- idempotency guards
- callback dedup
- polling
- payload versioning
- compensation
- reconciliation

JUST × 7

Workflow Engine v1.0

We didn’t set out to build a workflow engine.

We just kept solving the next requirement.

JUST × 7

```
class DefinitelyNotAWorkflow < ApplicationJob
  # 847 lines
end
```

Job vs workflow

A job

> "Please do this."

Send the email. Resize the image. Sync the record.

Short. Discrete. Retry as a unit.

A workflow

> "Make sure this eventually completes."

Onboarding. Fulfillment. Payment lifecycle.

State. Time. Events. Compensation.

It’s a spectrum. Nothing happens when you cross it. Rubocop has no rule for this.

A workflow is what a job gradually turns into.

Yes, these exist

```
Sidekiq Pro Batches · acidic_job · Gush
AASM · Statesman · GoodJob
```

Each one makes **a step** better.

None of them fundamentally changes who owns **keeping the process alive**.

What if keeping the process alive was the runtime’s job instead of ours?

Not the business logic. The machinery.

Durable execution

Several systems do this

`Temporal · DBOS · Restate · Inngest`

I picked one, for a boring reason that matters to this room:

the Ruby SDK went GA last October.

GA announced 1 Oct 2025 · temporal.io/changelog/ruby-sdk-generally-available

Three words

Workflow: decides what happens next. Your orchestration.

Activity: touches the outside world. Charge the card.

Worker: the process you run that executes both.

Everything else is day two.

The same process

```ruby
class OrderWorkflow < Temporalio::Workflow::Definition
  def execute(order_id)
    Temporalio::Workflow.execute_activity(ChargePayment, order_id)
    Temporalio::Workflow.execute_activity(ReserveInventory, order_id)

    Temporalio::Workflow.wait_condition { @warehouse_ready }

    Temporalio::Workflow.execute_activity(ShipOrder, order_id)
  end

  workflow_signal
  def warehouse_confirmed = @warehouse_ready = true
end
```

You already wrote all of this

What you built in the first half	Here
`order.charged?` guards	history records the activity completed
`CheckWarehouseJob` re-enqueueing itself	`wait_condition`
callback token + dedup table	`signal`
status column as program counter	the workflow's own position in the code
`ReconcileStuckOrdersJob`	—

Waiting became a language feature.

Who owns keeping this process alive?

The first half of this talk: **me**.

My columns. My scheduler. My dedup table. My reconciler. My phone.

Here: **the runtime**.

The same process, twice

Same requirements. Same five failures. Both broken on purpose.

Kill the worker, right after payment

Sidekiq

Retries. Hits my idempotency guards.

It works.

Because I wrote the guards.

Temporal

Another worker continues after the payment activity.

There is no recovery code to show you.

Same outcome. Different answer to *who is responsible for it*.

The callback fires twice

Sidekiq

Dedup table. Unique index.

A philosophical debate about what "twice" means.

Temporal

Workflow is already past that wait.

Second signal is a no-op.

Not magic. Somebody wrote that logic. The question is whether it was **you**, at 2am.

The 48-hour wait

Temporal’s test environment skips time.

On the Sidekiq side I faked the clock and fired the cron by hand.

I built a fake clock to test my fake timer.

Who owns what

Concern	Rails + Sidekiq	Durable runtime
Retry	Job system	Runtime
Timer	Scheduler + job	Runtime
Workflow state	Your DB	Runtime
External event	Callback + dedup	Signal
Recovery	**You**	**Runtime**
Orchestration	You	Workflow
Business logic	**You**	**You**

Okay. What did we just buy?

Determinism

```ruby
# Inside workflow code. Replay breaks all of these
Time.now
SecureRandom.uuid
rand(100)
Order.find(id)             # any IO at all

# What you write instead
Temporalio::Workflow.now
Temporalio::Workflow.random.uuid
Temporalio::Workflow.execute_activity(FetchOrder, id)
```

`Time.now`. `Order.find`. The two most ordinary things in Rails. Inside a workflow, they're bugs.

The rest of the bill

Another runtime. Not Rails, Postgres, Redis any more. Someone operates this.

Debugging changes shape. Not a stack trace. An execution history.

Versioning becomes first-class. Old executions in flight when you deploy.

Your organisation has one more thing it must understand. Every hire. Every incident.

Reduces *application* complexity. Increases *system* complexity. That's a real trade.

Where’s your line?

That’s a job

- short
- independent
- stateless
- retryable as a unit
- fire-and-forget

That might be a workflow

- multi-step
- long-running
- stateful
- waits on events or humans
- needs compensation
- **keeps accumulating orchestration code**

An abstraction is useful right up until you start implementing the abstraction it was supposed to give you.

That job you thought of when I asked at the start.

If there's a comment on it with someone's name, and that person doesn't work there any more,

Is that still a job?

So, where would you draw the line?

Anubhav Jain

x.com/anewbhav

linkedin.com/in/anew-bhav

Slides: anewbhav.dev/talks/sidekiq-workflow-engine

References & credits

Temporal: temporal.io · Ruby SDK GA announcement, 1 Oct 2025 · github.com/temporalio/sdk-ruby · docs.temporal.io/develop/ruby
Code on these slides verified against gem `temporalio` 1.6.0.

Sidekiq: Mike Perham / Contributed Systems · sidekiq.org · Batches (Pro)

Ruby workflow / job libraries named in this talkacidic_job · Gush · AASM · Statesman · GoodJob

Other durable execution runtimes: DBOS · Restate · Inngest

All example code is my own. The order-fulfillment scenario is illustrative, not taken from any employer's system. No affiliation with Temporal or any project listed.